

From Image Classification to Threat Reasoning: Building an End-to-End Aircraft Identification and Decision Support Pipeline



Overview

This project started with a simple question:

How can we improve aircraft identification: making it faster, more accurate, and enabling a more objective threat assessment—under operational time pressure??

Inspired by how naval and air combat information officers operate in real-world settings, the goal evolved into building a system that does more than just classify an image. Instead, it aims to:

- **Identify an aircraft from an image:** The type of aircraft.
- **Quantify confidence in that identification:** How sure are we that it is this aircraft.
- **Retrieve structured technical specifications:** What are the combat capabilities of this aircraft.
- **Reason about threat level under uncertainty:** Given the combat capabilities, how much of a threat does it pose to ourself.

- **Recommend sensible next actions:** What's the best course of action given all the information we have(includes **HUMAN** in the loop here)

This document walks through how that system was built end-to-end, including what went wrong along the way.

High-Level Pipeline

At a high level, the pipeline looks like this:

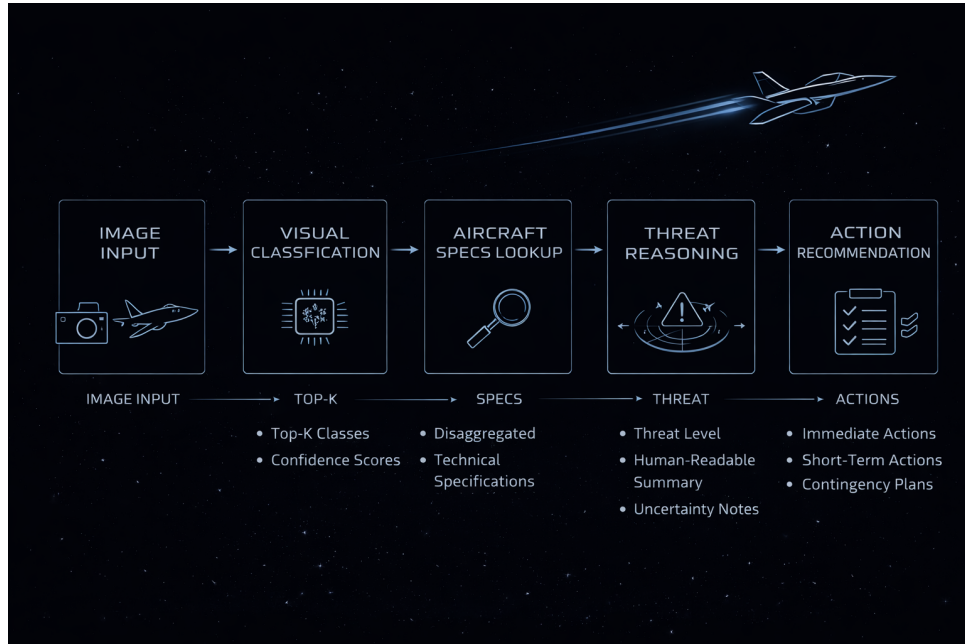


Figure 1: Operational Pipeline

Dataset and Early Challenges

Probably spent the most time over here (If not looped back and redid the data collection due to issues we will discuss later on). Used selenium to automate scraping from wikipedia headlessly because all the other options for looking up required api keys and unnecessary overcomplications and paywalls.

We collect the names of aircrafts and append it to a csv file and use the csv file then to collect the corresponding images after that to create our own dataset (*In my head it was perfect, however given that I've enumerated the list of Challenges tells you something doesn't it?*).

Challenge 1: Initial Wikipedia Scrapping

Approach: The initial data collection performed surface-level scraping of Wikipedia, extracting primary links and saving them to a CSV. This method was designed to avoid deep link navigation and an excessive number of dataset classes.

Drawback: This resulted in only high-level category links (e.g., "aircraft from the 1970s," "aircraft from the 1980s") without differentiating between list pages and individual aircraft pages.

Solution: Integrated Ollama into the pipeline to classify whether a given Wikipedia page represented a list of aircraft or a specific aircraft model.

Outcome: Gathered approximately 1700 aircraft classes. However, the dataset included significant noise from unrelated pages such as aircraft accidents and explosions.

Challenge 2: Revised Wikipedia Scraping for Dataset Cleaning

Approach: Refined the Ollama query to explicitly filter out noise entries like accidents, explosions, and incomplete aircraft pages.

Drawback: The more aggressive link crawling began including pages for **prototypes and experimental weapons**, which were not desired for the target operational dataset.

Solution: Enhanced the Ollama filter to also exclude prototypes and experimental aircraft.

Outcome: Reduced the dataset to roughly 1300 aircraft classes, though it still contained **non-operational aircraft**.

Challenge 3: Image Collection

Approach: Used a naive method to gather images from common sources like Wikimedia and Google search results.

Drawback: The collected images were often irrelevant (non-aircraft), with a high prevalence of **accident and explosion scenes**.

Solution: Implemented a soft filtering step using HTML scraping and the *alt* text from page sources to better identify relevant images.

Outcome: Successfully collected images for approximately 800 aircraft classes.

Challenge 4: Insufficient Images per Class

Approach: Many aircraft classes, particularly from the early aviation era, had few or no available images due to limited documentation.

Drawback: This scarcity drastically reduced the usable number of classes for model training.

Solution: Applied a minimum image threshold and removed all aircraft classes below it, as preliminary models trained on sparse data performed poorly.

Outcome: Final dataset contained approximately **35 viable aircraft classes**.

Challenge 5: Final Data Quality and Structural Integrity

Approach: A final audit of the dataset revealed that one class (PN-1) contained significant noise and irrelevant images.

Problem: Removing this class altered the label space, which caused a classifier head dimension mismatch in the trained model.

Solution: The PN-1 class was purged, necessitating a complete model retrain from scratch.

Outcome: This resulted in a final, clean dataset of 34 aircraft classes. The process underscored a critical lesson: any change to the label space has cascading effects and requires full retraining.

Once the dataset was fully vetted, it was then split 80:20 into train and val and metrics were measured for various models.

Model Training

Several architectures were evaluated, including metric-learning approaches and transformer-based models. Ultimately, a fine-tuned EfficientNet variant provided the best balance between performance, stability, and inference speed for this use case.

Key techniques used:

- **Class-balanced loss:** Adjusts the loss function to account for imbalanced class distributions, preventing the model from being biased toward the majority classes.
- **Label smoothing** – Replaces hard one-hot labels with smoothed targets (e.g., 0.9 for the true class, 0.1 for others) to reduce model overconfidence and improve generalization.
- **Freeze / unfreeze scheduling** – Initially freezes the backbone layers to stabilize early training, then progressively unfreezes them to allow fine-tuning of deeper features.
- **Explicit overfitting diagnostics (train–val gap tracking)** – Systematically monitors the performance gap between training and validation sets to detect overfitting early and adjust regularization or training duration.

Despite achieving high training accuracy early, validation accuracy plateaued around 40%. This gap was not ignored; it became a design constraint that influenced how much trust downstream reasoning should place on the classifier.

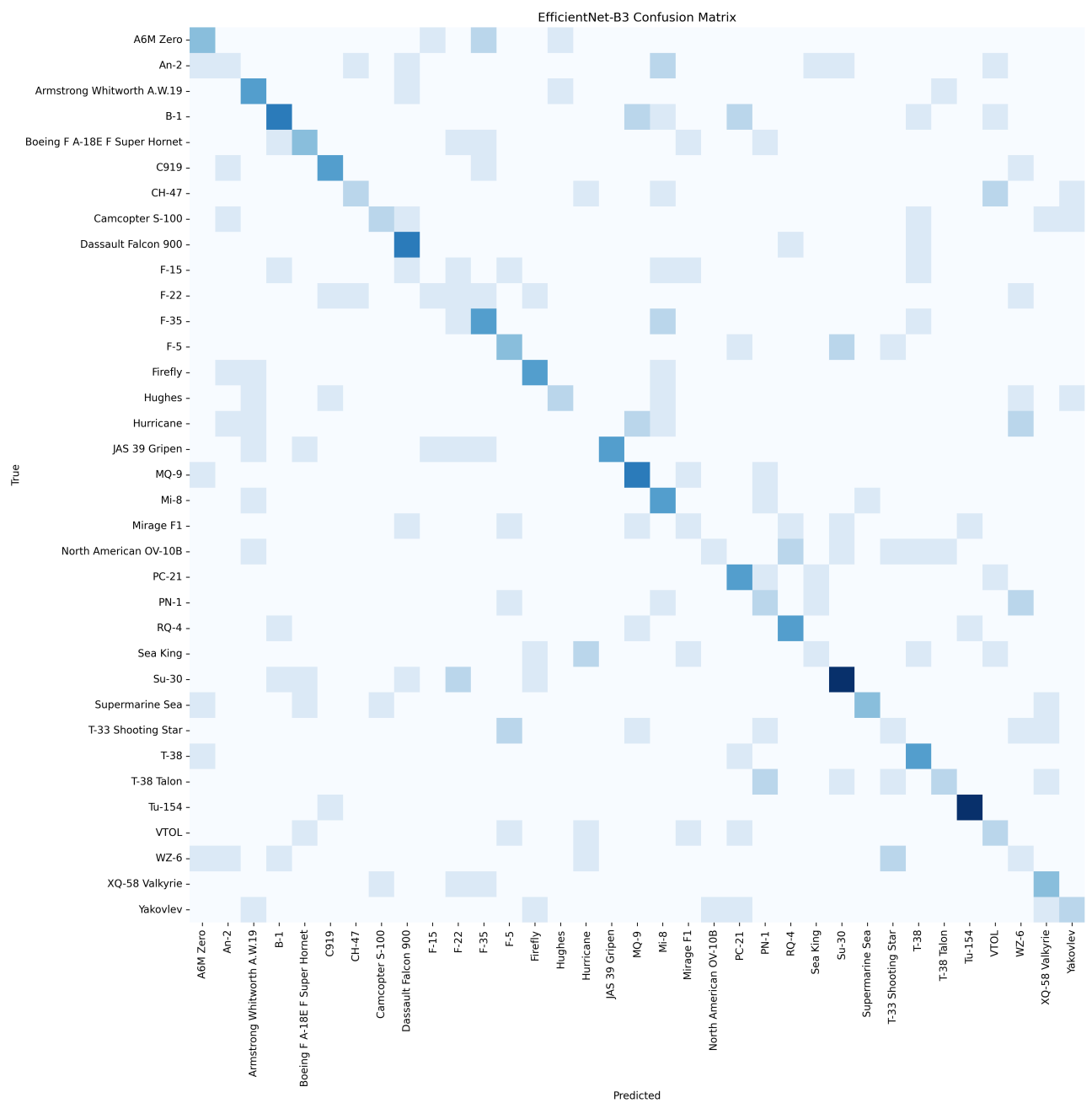


Figure 2: EfficientNet Confusion Matrix

Output

```
=====
TOP-1 IDENTIFICATION
Aircraft: F-15
Confidence: 89.66%

Threat Assessment
Level: High
F-15 is assessed as a high threat. It demonstrates high speed (~2450 km/h), beyond-visual-range engagement, strike capability, electronic warfare systems. This aircraft should be treated as a cr

Recommended Actions

IMMEDIATE:
- Maintain continuous tracking
- Cross-check with radar and IFF

SHORT TERM:
- Increase sensor fusion priority
- Assign interceptor-ready assets
- Elevate local alert posture

CONTINGENCY:
- Prepare escalation protocols

Uncertainty Notes:
- Assessment confidence is high
```

Figure 3: Output for F22 Raptor

Note that this is the result for an F-22 raptor which got classified as an F-15 which is a different aircraft but since they're from the same family we'll take this as a win. Not to mention the fact that we only achieved 41% accuracy with the model. We conclude that model performance cannot be improved further under the current conditions, as we are fundamentally limited by the small size and narrow scope of the available training data.

Top-K Classification Instead of Top-1

Rather than relying on a single predicted class, the inference pipeline uses a Top-K approach. This decision was deliberate:

- Real images are ambiguous
- Confidence matters more than correctness alone
- Downstream systems should degrade gracefully

Each Top-K output includes:

- Aircraft name
- Confidence score

Only structurally valid predictions are accepted, preventing silent failures from malformed outputs.

Aircraft Specification Database

To decouple vision from domain knowledge, aircraft specifications were extracted and stored separately.

For each aircraft:

- A dedicated YAML file is generated

- Units are normalized numerically
- Capabilities are expressed as boolean flags

This design allows the reasoning layer to remain model-agnostic and extensible.

```
def ask_ollama(spec_text):
    prompt = f"""
You are a military aviation analyst.

From the provided information, produce STRICT YAML ONLY with this schema:

aircraft:
  name:
  role:
  country:
  introduction_year:

performance:
  max_speed:
    raw:
    kph:
  combat_range:
    raw:
    km:
  service_ceiling_m:

capabilities:
  stealth: true/false
  bvr_combat: true/false
  multirole: true/false
  strike: true/false
  electronic_warfare: true/false
Q
sensors:
  radar:
    present: true/false
    type:
  eo_ir: true/false
  datalink: true/false

armament:
  air_to_air:
    - name
  air_to_ground:
    - name
  internal_bay: true/false

threat_assessment:
  level: Low/Medium/High
  rationale:
```

Figure 4: Ollama Prompt for YAML

Threat Reasoning Under Uncertainty

One of the most important lessons was that **missing data is normal**.

Specifications may be incomplete. Classification confidence may be low. Some aircraft may be poorly documented.

Instead of failing, the threat reasoning module:

- Explicitly tracks uncertainty
- Adjusts threat labels based on confidence
- Emits explanatory summaries instead of raw scores

Threat levels are computed using interpretable heuristics such as speed, armament, and mission profile, not learned black-box scores.

Each assessment includes:

- Threat level
- Human-readable summary
- Uncertainty notes

This mirrors how real analysts communicate risk.

Action Recommendation Layer

On top of threat assessment sits an action recommender. This layer does not issue commands; it suggests sensible next steps based on doctrine-inspired logic.

Recommendations are grouped into:

- Immediate actions
- Short-term actions
- Contingency preparations

For example, a high-confidence, high-threat identification may trigger suggestions such as continuous tracking, sensor fusion prioritization, or interceptor readiness.

Code Used:

```
1 def recommend_actions(threat_level, confidence):
2     """
3     Returns recommended actions based on threat level and confidence.
4     """
5
6     actions = {
7         "immediate": [],
8         "short_term": [],
9         "contingency": []
10    }
11
```



```

12     # --- Baseline monitoring ---
13     actions["immediate"].append("Maintain continuous tracking")
14     actions["immediate"].append("Cross-check with radar and IFF")
15
16     if threat_level in ["Moderate", "High", "Severe"]:
17         actions["short_term"].append("Increase sensor fusion priority")
18         actions["short_term"].append("Assign interceptor-ready assets")
19
20     if threat_level in ["High", "Severe"]:
21         actions["short_term"].append("Elevate local alert posture")
22         actions["contingency"].append("Prepare escalation protocols")
23
24     if threat_level == "Severe":
25         actions["contingency"].append("Notify command authority")
26         actions["contingency"].append("Review rules of engagement")
27
28     # --- Confidence modifier ---
29     if confidence < 0.5:
30         actions["immediate"].append("Revalidate classification with
31             additional sensors")
32
33     return actions

```

Why This Matters

This project is not about achieving state-of-the-art accuracy.

It is about:

- Designing systems that reason under uncertainty
- Respecting the limits of machine perception
- Building pipelines that support humans, not replace them

The final system behaves less like a classifier and more like a junior analyst: confident when warranted, cautious when uncertain, and always explicit about its assumptions.

Next Steps

Potential future extensions include:

- **Temporal tracking across multiple frames:** Extending the system from single-image inference to temporal analysis would enable persistent tracking of aircraft across consecutive frames or sensor updates. This would allow the system to reason about motion patterns, trajectory changes, loitering behavior, and intent inference, reducing susceptibility to transient misclassifications and improving confidence stability over time.
- **Multi-aircraft fusion:** Supporting simultaneous detection and reasoning over multiple aircraft would allow the system to assess coordinated behavior, formation flying, and

swarm-like activity. This is particularly relevant in contested airspace, where threat assessment depends not only on individual platforms but also on collective behavior and relative positioning.

- **Rules-of-engagement-aware recommendations:** Integrating configurable rules of engagement (ROE) using Retrieval Augmented Generation (RAG) systems would constrain threat assessments and action recommendations within predefined operational, legal, and political boundaries. This ensures that system outputs remain aligned with mission-specific doctrine and reduces the risk of inappropriate escalation suggestions.
- **Confidence-calibrated decision thresholds:** Incorporating calibrated confidence thresholds would allow the system to explicitly modulate its outputs based on uncertainty. Low-confidence classifications could trigger conservative recommendations or defer judgment, while high-confidence assessments could enable stronger advisory signals, reinforcing appropriate caution in ambiguous scenarios.

Ethical Implications

The integration of an automated identification and threat assessment system introduces significant ethical considerations centered on human judgment and accountability.

- **Automation Bias and Conformity:** Presenting a system-generated assessment creates **automation bias**, predisposing operators to conform to its suggestions. This can lead toward either unwarranted escalation or passive acceptance of potentially inaccurate judgments. This risk is amplified by the tendency of LLM-based components to align with user input, potentially reinforcing pre-existing biases rather than providing a neutral evaluation.
- **Desensitization and Decision Pressure:** Automating the threat-response loop can desensitize commanding officers to the gravity of their decisions, placing them under intense pressure to make rapid, system-validated calls. This environment risks eroding the crucial **tacit knowledge** and **command intuition** developed through experience.
- **The Fallibility Principle:** All automated systems are fallible. Therefore, the system must be designed strictly as a **decision-support tool**—a guiding hand that provides critical information without usurping the commander’s ultimate responsibility. The human must remain firmly **in the decision loop** and **at the helm**.

Closing Thoughts

Building this system required as much systems thinking as machine learning. Most of the work happened not in improving accuracy, but in making failures understandable and outputs actionable.

That, more than any metric, was the real objective.